

- PyTorch для начинающих: Классификация цифр на MNIST

Этот блокнот служит введением в PyTorch, демонстрируя процесс создания, обучения и оценки моделей глубокого обучения для задачи классификации изображений. Мы рассмотрим:

- Основные принципы PyTorch, включая тензоры, слои и оптимизаторы.
- Работу с данными: загрузка датасета MNIST, предварительная обработка и создание загрузчиков данных.
- Построение и обучение простых полносвязных нейронных сетей (Sequential API).
- Создание более сложных сверточных нейронных сетей (Class-based API).
- Оценку производительности модели с использованием метрик и матрицы ошибок.

Базированный import

```
import torch # Основная библиотека PyTorch, предоставляющая тензорные операции.
import torch.nn as nn # Содержит модули и классы для построения нейронных сетей.
import torch.nn.functional as F # Предоставляет функциональные интерфейсы для операций нейронных сетей.
import torch.optim as optim # Реализует различные алгоритмы оптимизации.
import torch.utils.data as data # Утилиты для загрузки и предварительной обработки данных.

import torchvision.transforms as transforms # Предоставляет общие преобразования изображений.
import torchvision.datasets as datasets # Предлагает популярные наборы данных (например, MNIST).

from torchsummary import summary # Утилита для получения сводки по модели PyTorch.
from sklearn import metrics # Используется для расчета различных метрик производительности.

from tqdm.notebook import trange, tqdm # Для отображения прогресс-баров в ноутбуках.
import matplotlib.pyplot as plt # Библиотека для построения графиков и визуализации данных.
import numpy as np # Библиотека для численных операций.

import copy # Используется для создания копий объектов.
import random # Для генерации случайных чисел.
import time # Для операций, связанных со временем.
```

Чтобы гарантировать воспроизводимые результаты, мы устанавливаем случайное начальное число для Python, Numpy и PyTorch

```
def torch_seed(seed = 42, deterministic = True):
    random.seed(seed) # Устанавливаем seed для модуля random Python.
    np.random.seed(seed) # Устанавливаем seed для NumPy.
    torch.manual_seed(seed) # Устанавливаем seed для CPU PyTorch.
    torch.cuda.manual_seed(seed) # Устанавливаем seed для GPU PyTorch (если используется).
    torch.backends.cudnn.deterministic=deterministic # Делаем работу CuDNN детерминированной для воспроизводимости результатов.
```

```
torch_seed(seed = 42, deterministic = True) # Вызываем функцию для установки всех seed.
```

```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu') # Определяем, будет ли использоваться GPU (cuda) или CPU.
print('Считаем нейронки на:', device) # Выводим выбранное устройство.
```

Считаем нейронки на: cuda

Грузим данные

```
train_data = datasets.MNIST(root='./data', # Указываем папку для сохранения данных.
                             train=True, # Загружаем тренировочную часть датасета.
                             download=True, # Если данных нет, скачиваем их.
                             transform=None) # Пока не применяем никаких преобразований.
train_data # Выводим информацию о загруженных данных.
```

```
100% |#####| 9.91M/9.91M [00:00<00:00, 20.5MB/s]
100% |#####| 28.9k/28.9k [00:00<00:00, 592kB/s]
100% |#####| 1.65M/1.65M [00:00<00:00, 4.67MB/s]
100% |#####| 4.54k/4.54k [00:00<00:00, 10.7MB/s]
Dataset MNIST
Number of datapoints: 60000
Root location: ./data
Split: Train
```

Изображения скачиваются в формате PNG

```
train_data[0] # Проверяем формат первого элемента: изображение (PIL.Image) и метка класса.
```

```
(<PIL.Image.Image image mode=L size=28x28>, 5)
```

Исходные значения - 8-битные `int`, как обычно

```
train_data.data[0] # Проверяем данные самого изображения в виде тензора (пиксели).
```

[illegible]



```
0, 16, 93, 252, 253, 187, 0, 0, 0, 0, 0, 0, 0],
[ 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
0, 0, 0, 249, 253, 249, 64, 0, 0, 0, 0, 0, 0],
[ 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
46, 130, 183, 253, 253, 207, 2, 0, 0, 0, 0, 0, 0],
[ 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 39, 148,
229, 253, 253, 253, 250, 182, 0, 0, 0, 0, 0, 0, 0],
[ 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 24, 114, 221, 253,
253, 253, 253, 201, 78, 0, 0, 0, 0, 0, 0, 0, 0],
[ 0, 0, 0, 0, 0, 0, 0, 0, 23, 66, 213, 253, 253, 253,
253, 198, 81, 2, 0, 0, 0, 0, 0, 0, 0, 0, 0],
[ 0, 0, 0, 0, 0, 0, 18, 171, 219, 253, 253, 253, 253, 195,
80, 9, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
[ 0, 0, 0, 0, 55, 172, 226, 253, 253, 253, 253, 244, 133, 11,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
[ 0, 0, 0, 0, 136, 253, 253, 253, 212, 135, 132, 16, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
[ 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0],
[ 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0],
[ 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0],
[ 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0],
dtype=torch.uint8)
```

Вместо простой нормализации в диапазон от 0 до 1 некоторые любят приводить к нулевому среднему и единичной дисперсии

Для этого посчитаем среднее и ско для тренировочных данных

```
mean = train_data.data.float().mean() / 255 # Вычисляем среднее значение пикселей для нормализации.
std = train_data.data.float().std() / 255 # Вычисляем стандартное отклонение пикселей для нормализации.
```

```
print(f'Посчитанное среднее: {mean}') # Выводим посчитанное среднее.
print(f'Посчитанное СК0: {std}') # Выводим посчитанное стандартное отклонение.
```

```
Посчитанное среднее: 0.13066047430038452
Посчитанное СК0: 0.30810779333114624
```

В PyTorch очень любят делать преобразователи, которые взаимодействуют прямо с загрузкой

Умные люди добавляют в `transforms` много интересного, но мы пока ограничимся:

- `ToTensor()` - преобразованием данных в тензорный формат от PyTorch (а ещё "бесплатно" преобразуют в диапазон от 0 до 1);
- `Normalize` - нормализация данных (вычитание среднее, деление на указанную дисперсию);

При этом объект `Compose` должен вам напомнить `Pipeline` из scikit-learn, где мы последовательно объединяем преобразования

```
train_transforms = transforms.Compose([ # Создаем последовательность преобразований для тренировочных данных.
    transforms.ToTensor(), # Преобразует PIL Image в Tensor и масштабирует значения пикселей из [0, 255] в [0.0, 1.0].
    transforms.Normalize(mean=[mean], std=[std]) # Нормализует тензор, вычитая среднее и деля на стандартное отклонение.
])

test_transforms = transforms.Compose([ # Создаем такую же последовательность преобразований для тестовых данных.
    transforms.ToTensor(),
    transforms.Normalize(mean=[mean], std=[std])
])
```

Перезагрузим данные, но уже с учетом преобразования

```
train_data = datasets.MNIST(root='./data',
                             train=True,
                             download=True,
                             transform=train_transforms) # Перезагружаем тренировочные данные, применяя определенные преобразования.

test_data = datasets.MNIST(root='./data',
                            train=False, # Загружаем тестовую часть датасета.
                            download=True,
                            transform=test_transforms) # Перезагружаем тестовые данные, применяя определенные преобразования.
```

Делаем загрузчик данных `DataLoader`

Это итератор, который будет поставлять отдельные батчи в процедуру обучения

Для тренировочных данных мы также реализуем процедуру перемешивания, чтобы при каждой эпохе обучения батчи были разными

Для тестовых данных - не принципиально

Размер батча - как правило "сколько влезает в память без последствий"

Но любят степени двойки

```
BATCH_SIZE = 64 # Определяем размер батча (количество изображений, обрабатываемых за один шаг).

train_iterator = data.DataLoader(train_data,
                                 shuffle=True, # Перемешиваем данные в каждом "проходе" (эпохе) для лучшего обучения.
                                 batch_size=BATCH_SIZE) # Указываем размер батча для тренировочного загрузчика.

test_iterator = data.DataLoader(test_data,
                                batch_size=BATCH_SIZE) # Создаем загрузчик для тестовых данных, без перемешивания.
```

Простая Полносвязная Нейронная сеть

Как и в TensorFlow библиотека PyTorch имеет несколько API для создания моделей

Начнём с простого - последовательный (`Sequential`). Идея максимально близкая к тому что было в TensorFlow

Ключевые различия

- полносвязный слой теперь называется `Linear`
- в нем необходимо указывать размер входа и размер выхода
- функции активации применяются отдельно

```
model = nn.Sequential(
    nn.Flatten(), # Выпрямляет входной тензор, преобразуя его из 2D изображения в 1D вектор для полносвязных слоев.
    nn.Linear(784, 16), # Первый полносвязный слой с 784 входными признаками (28*28) и 16 выходными нейронами.
    nn.Sigmoid(), # Функция активации Sigmoid, применяемая к выходу первого полносвязного слоя.
    nn.Linear(16, 16), # Второй полносвязный слой с 16 входными и 16 выходными нейронами.
    nn.Sigmoid(), # Функция активации Sigmoid, применяемая к выходу второго полносвязного слоя.
    nn.Linear(16, 10) # Выходной полносвязный слой с 16 входными и 10 выходными нейронами (по одному для каждого класса).
```



```
)

model = model.to(device) # Перемещаем нашу модель на выбранное устройство (GPU или CPU).
```

Структура здесь рисуется не через метод, а через функцию `summary()`

В которой просят указать размерность входных данных

В формате "количество каналов, ширина, высота"

```
summary(model, input_size = (1,28,28)) # Выводим сводку по модели, указывая размер входных данных (каналы, высота, ширина).
```

Layer (type)	Output Shape	Param #
Flatten-1	[-1, 784]	0
Linear-2	[-1, 16]	12,560
Sigmoid-3	[-1, 16]	0
Linear-4	[-1, 16]	272
Sigmoid-5	[-1, 16]	0
Linear-6	[-1, 10]	170

```
Total params: 13,002
Trainable params: 13,002
Non-trainable params: 0

Input size (MB): 0.00
Forward/backward pass size (MB): 0.01
Params size (MB): 0.05
Estimated Total Size (MB): 0.06
```

При инициализации модели сразу генерируются случайные веса

Так что формально можно "запустить" модель и так, на случайных данных

```
X = torch.randn(1, 1, 28, 28).to(device) # Создаем случайный входной тензор размером (батч, каналы, высота, ширина) и перемещаем его на устройство.
out = model(X) # Пропускаем случайный тензор через модель, чтобы получить предсказания.
print(out.data.cpu().numpy(), '\n На барабане Цифра:', out.argmax().item()) # Выводим логиты и предсказанный класс (цифру).
```

```
[[-0.92749465 -0.18728289  0.16623098 -0.2872058  0.25391015  0.56337595
  0.14742337  0.04841586 -0.09635319  0.49248257]]
На барабане Цифра: 5
```

Подготовка к обучению модели

Как и у TensorFlow нам необходимо задать оптимизатор, функцию потерь и метрики, которые мы считаем

Напомним что оптимизатор - это то, по какому правилу обновляются параметры модели в ходе обучения

Задаются объектом соответствующего [класса](#)

Сделаем оптимизатор таким же как и был в TensorFlow по умолчанию

```
optimizer = optim.SGD(model.parameters(), lr=0.01) # Инициализируем оптимизатор SGD (Stochastic Gradient Descent) с заданной скоростью обучения (learning rate).
optimizer

SGD (
  Parameter Group 0
    dampening: 0
    differentiable: False
    foreach: None
    fused: None
    lr: 0.01
    maximize: False
    momentum: 0
    nesterov: False
    weight_decay: 0
)
```

Затем нам нужна функция потерь, которая выдает числеку тем больше, чем хуже наша модель

Пусть будет стандартная кросс-энтропия

```
criterion = nn.CrossEntropyLoss() # Определяем функцию потерь как кросс-энтропию, подходящую для задач классификации.
```

Функцию потерь лучше бы тоже переместить на тоже устройство, где у нас модель

```
criterion = criterion.to(device) # Перемещаем функцию потерь на то же устройство, что и модель.
```

А вот метрики как правило пишут самостоятельно

Например ниже пример функции расчёта метрики доля правильных ответов

```
def calculate_accuracy(y_pred, y):
    top_pred = y_pred.argmax(1, keepdim=True) # Находим индекс класса с максимальной предсказанной вероятностью.
    correct = top_pred.eq(y.view_as(top_pred)).sum() # Сравниваем предсказанные классы с истинными и считаем количество правильных предсказаний.
    acc = correct.float() / y.shape[0] # Вычисляем долю правильных ответов.
    return acc
```

Обучение

В PyTorch принято цикл обучения как правило записывают в отдельную функцию

Можно больше кастомизировать по себя

Однако как правило вы будете использовать что-то похожее почти всегда

```
def train(model, iterator, optimizer, criterion, device):

    # обнуляем метрики и функцию потерь
    epoch_loss = 0
    epoch_acc = 0

    # переводим модель в режим обучения
    model.train()

    # перебираем батчи из нашего итератора
    for (x, y) in tqdm(iterator, desc="Учим", leave=False):
        # всё складываем на одно устройство
```



```
x = x.to(device)
y = y.to(device)

# обнуляем градиенты (каждый батч с нового листа)
optimizer.zero_grad()

# делаем предсказание текущей модели
y_pred = model(x)

# оцениваем функцию потерь
loss = criterion(y_pred, y)

# оцениваем метрику
acc = calculate_accuracy(y_pred, y)

# тык и посчитали производные
loss.backward()
# тык и обновили веса по правилам оптимизатора
optimizer.step()

# добавили текущие значения функции потерь и метрики
epoch_loss += loss.item()
epoch_acc += acc.item()
# вернули средние
return epoch_loss / len(iterator), epoch_acc / len(iterator)
```

Оценка

Обычно отдельно описывают функцию для оценки (на валидационных данных)

```
def evaluate(model, iterator, criterion, device):

    # обнуляем метрики и функцию потерь
    epoch_loss = 0
    epoch_acc = 0

    # переводим модель в режим оценки
    model.eval()
    # даём всем понять что мы не будем считать градиенты. Вообще
    with torch.no_grad():
    # перебираем батчи из нашего итератора
        for (x, y) in tqdm(iterator, desc="Оцениваем", leave=False):
            # всё складываем на одно устройство
            x = x.to(device)
            y = y.to(device)

            # делаем предсказание текущей модели
            y_pred = model(x)

            # оцениваем функцию потерь
            loss = criterion(y_pred, y)

            # оцениваем метрику
            acc = calculate_accuracy(y_pred, y)

            # добавили текущие значения функции потерь и метрики
            epoch_loss += loss.item()
            epoch_acc += acc.item()
    # вернули средние
    return epoch_loss / len(iterator), epoch_acc / len(iterator)
```

Просто функция для оценки времени

```
def epoch_time(start_time, end_time):
    elapsed_time = end_time - start_time
    elapsed_mins = int(elapsed_time / 60)
    elapsed_secs = int(elapsed_time - (elapsed_mins * 60))
    return elapsed_mins, elapsed_secs
```

Тренировка модели

Поехали обучать

Сделаем столько же эпох, сколько и в примере с TensorFlow

Также научимся сохранять лучший результат

```
EPOCHS = 10

best_valid_loss = float('inf')

for epoch in trange(EPOCHS):

    start_time = time.monotonic()

    # обучаем на тренировочных данных
    train_loss, train_acc = train(model, train_iterator, optimizer, criterion, device)
    # оцениваем на валидационных
    valid_loss, valid_acc = evaluate(model, test_iterator, criterion, device)

    # сохраняем лучшую модель по потерям на валидационной выборке
    if valid_loss < best_valid_loss:
        best_valid_loss = valid_loss
        torch.save(model.state_dict(), 'tut1-model.pt')

    end_time = time.monotonic()

    epoch_mins, epoch_secs = epoch_time(start_time, end_time)

    print(f'Epoch: {epoch+1:02} | Epoch Time: {epoch_mins}m {epoch_secs}s')
    print(f'\tTrain Loss: {train_loss:.3f} | Train Acc: {train_acc*100:.2f}%')
    print(f'\t Val. Loss: {valid_loss:.3f} | Val. Acc: {valid_acc*100:.2f}%')
```



100%

10/10 [03:29<00:00, 20.31s/it]

Epoch: 01 | Epoch Time: 0m 25s
Train Loss: 2.294 | Train Acc: 14.54%
Val. Loss: 2.262 | Val. Acc: 18.55%

Epoch: 02 | Epoch Time: 0m 20s
Train Loss: 2.231 | Train Acc: 26.40%
Val. Loss: 2.187 | Val. Acc: 31.77%

Epoch: 03 | Epoch Time: 0m 19s
Train Loss: 2.119 | Train Acc: 38.85%
Val. Loss: 2.032 | Val. Acc: 43.68%

Epoch: 04 | Epoch Time: 0m 23s
Train Loss: 1.930 | Train Acc: 45.41%
Val. Loss: 1.822 | Val. Acc: 44.97%

Epoch: 05 | Epoch Time: 0m 19s
Train Loss: 1.724 | Train Acc: 50.77%
Val. Loss: 1.628 | Val. Acc: 54.79%

Загружаем лучшую модель

```
model.load_state_dict(torch.load('tut1-model.pt'))

test_loss, test_acc = evaluate(model, test_iterator, criterion, device)
print(f'\t Val. Loss: {test_loss:.3f} | Val. Acc: {test_acc*100:.2f}%')
```

Val. Loss: 1.020 | Val. Acc: 75.36%

Epoch: 06 | Epoch Time: 0m 20s
Train Loss: 1.394 | Train Acc: 62.66%
Val. Loss: 1.327 | Val. Acc: 64.35%

Отдельно сделаем функцию для предсказания результатов

В которой перебираем все изображения и сохраняем предсказания в виде вероятности

Epoch: 08 | Epoch Time: 0m 19s

```
def get_predictions(model, iterator, device):
    # переводим модель в режим оценки
    model.eval()

    images = []
    labels = []
    probs = []

    # даём всем понять что мы не будем считать градиенты. Вообще
    with torch.no_grad():

        for (x, y) in iterator:
            # всё складываем на одно устройство
            x = x.to(device)

            # делаем предсказание текущей модели
            y_pred = model(x)

            # делаем софт-макс преобразование
            y_prob = F.softmax(y_pred, dim=-1)

            images.append(x.cpu())
            labels.append(y.cpu())
            probs.append(y_prob.cpu())

    images = torch.cat(images, dim=0)
    labels = torch.cat(labels, dim=0)
    probs = torch.cat(probs, dim=0)

    return images, labels, probs
```

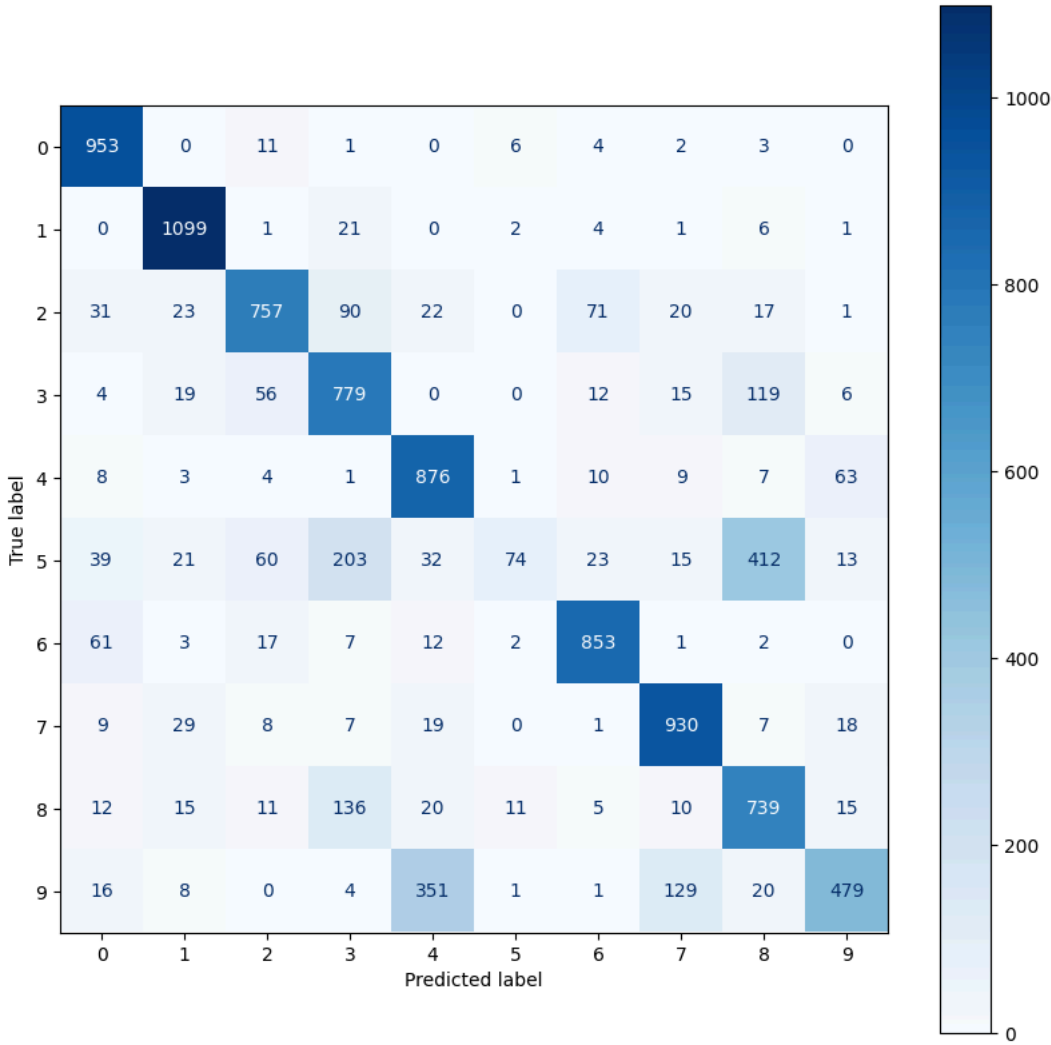
```
images, labels, probs = get_predictions(model, test_iterator, device)
# находим предсказание модели как аргмакс вероятности
pred_labels = torch.argmax(probs, 1)
```

Матрица ошибок

```
def plot_confusion_matrix(labels, pred_labels):

    fig = plt.figure(figsize=(10, 10))
    ax = fig.add_subplot(1, 1, 1)
    cm = metrics.confusion_matrix(labels, pred_labels)
    cm = metrics.ConfusionMatrixDisplay(cm, display_labels=range(10))
    cm.plot(values_format='d', cmap='Blues', ax=ax)
```

```
plot_confusion_matrix(labels, pred_labels)
```



А вот TensorFlow с девятками при прочих равных не запутывался...

Свёрточная Нейронная сеть

Заодно посмотрим как делать нейронную сеть в виде класса

По сути всё сводится к "созданию" набора слоёв, которые потом связываются как в функциональном API у TensorFlow.

Ключевые отличия и важные моменты при создании модели как класса `nn.Module`:

- Наследование от `nn.Module`:** Все нейронные сети в PyTorch, будь то простые слои или сложные архитектуры, наследуются от базового класса `torch.nn.Module`. Это предоставляет необходимую функциональность для работы с параметрами модели, перемещения на устройство (GPU/CPU) и т.д.
- Метод `__init__`:** В конструкторе `__init__` определяются все слои и компоненты вашей нейронной сети, которые будут иметь обучаемые параметры (например, `nn.Conv2d`, `nn.Linear`). Здесь вы создаете экземпляры этих слоев.
- Метод `forward`:** Этот метод определяет, как данные проходят через вашу сеть, то есть описывает логику прямого распространения. Здесь вы "соединяете" слои, определенные в `__init__`, и применяете к ним функции активации, операции пулинга и другие преобразования.
- Размерности полносвязных слоев:** В отличие от `nn.Sequential` с `Flatten()`, при ручном определении архитектуры класса для полносвязных слоев (`nn.Linear`) вам часто приходится вручную рассчитывать размер входных признаков после всех сверточных слоев и пулинга. Это требует понимания того, как изменяются размерности тензоров после каждой операции. Например, `16 * 4 * 4` означает, что после сверток и пулинга у нас 16 каналов, каждый размером 4x4 пикселя.

Ну и опять для полносвязных слоёв просят напрямую посчитать количество весов

```
class NotLeNet(nn.Module):
    # Класс NotLeNet наследуется от nn.Module, базового класса для всех нейронных сетей в PyTorch.

    def __init__(self, in_channels, n_classes):
        # Конструктор класса, где определяются все слои и компоненты сети.
        super().__init__() # Вызов конструктора родительского класса.

        self.in_channels = in_channels # Количество входных каналов (например, 1 для MNIST).
        self.n_classes = n_classes # Количество классов для классификации (например, 10 для MNIST).

        # Первый сверточный слой:
        # in_channels: количество входных каналов (1 для черно-белого изображения).
        # out_channels: количество выходных каналов (фильтров), 6 в данном случае.
        # kernel_size: размер сверточного ядра (5x5).
        self.conv1 = nn.Conv2d(in_channels = self.in_channels,
                                out_channels = 6,
                                kernel_size = 5)

        # Второй сверточный слой:
        # in_channels: количество входных каналов, равное out_channels предыдущего слоя (6).
        # out_channels: количество выходных каналов (фильтров), 16 в данном случае.
        # kernel_size: размер сверточного ядра (5x5).
        self.conv2 = nn.Conv2d(in_channels = 6,
                                out_channels = 16,
                                kernel_size = 5)

        # Слой усредняющего пулинга (AvgPool2d):
        # kernel_size: размер окна пулинга (2x2).
        # stride: шаг перемещения окна пулинга (2), что уменьшает размерность в 2 раза по каждой оси.
        self.pool = nn.AvgPool2d(kernel_size = 2,
                                    stride = 2)

        # Первый полносвязный слой:
        # in_features: количество входных признаков. Рассчитывается как 16 каналов * 4x4 размерности карты признаков после сверток и пулинга.
        # out_features: количество выходных нейронов (16).
        self.fc1 = nn.Linear(in_features = 16 * 4 * 4,
                                out_features = 16)

        # Выходной полносвязный слой (для классификации):
        # in_features: количество входных признаков (16, из предыдущего полносвязного слоя).
        # out_features: количество классов (10).
        self.fc_out = nn.Linear(in_features = 16,
                                    out_features = self.n_classes)
```




```
def flatten(self, x):
    # Вспомогательная функция для "выпрямления" тензора (преобразования его из многомерного в одномерный вектор).
    # Это необходимо перед подачей данных в полносвязные слои после сверточных.
    # -1 означает, что PyTorch автоматически вычислит размер батча.
    return x.view(-1, 16 * 4 * 4)

def forward(self, x):
    # Метод forward определяет прямой проход данных через сеть.
    # Он описывает, как слои, определенные в __init__, соединены между собой.

    # Проход через первый сверточный слой, функцию активации Sigmoid и пулинг.
    x = self.conv1(x)
    x = torch.sigmoid(x) # Применяем сигмоидную функцию активации.
    x = self.pool(x)

    # Проход через второй сверточный слой, функцию активации Sigmoid и пулинг.
    x = self.conv2(x)
    x = torch.sigmoid(x) # Применяем сигмоидную функцию активации.
    x = self.pool(x)

    # Выпрямление тензора перед подачей в полносвязные слои.
    x = self.flatten(x)

    # Проход через первый полносвязный слой и функцию активации Sigmoid.
    x = self.fc1(x)
    x = torch.sigmoid(x) # Применяем сигмоидную функцию активации.

    # Проход через выходной полносвязный слой.
    # Здесь не применяется функция активации, так как CrossEntropyLoss ожидает "сырые" logits.
    x = self.fc_out(x)

    return x
```

Создадим модель (надо указать количество входных каналов и количество классов)

Также выберем другой оптимизатор (Adam), как мы делали в TensorFlow

```
LR = 0.001 # Устанавливаем скорость обучения (Learning Rate) для оптимизатора.
model_cnn = NotLeNet(1,10) # Создаем экземпляр нашей сверточной нейронной сети NotLeNet с 1 входным каналом и 10 классами.
criterion = nn.CrossEntropyLoss() # Определяем функцию потерь (Cross-Entropy Loss), подходящую для многоклассовой классификации.
metric = calculate_accuracy # Используем ранее определенную функцию для расчета метрики точности.
optimizer = torch.optim.Adam(model_cnn.parameters(), lr=LR) # Инициализируем оптимизатор Adam, передавая ему параметры модели и скорость обучения.

model_cnn = model_cnn.to(device) # Перемещаем модель на выбранное устройство (GPU/CPU).
criterion = criterion.to(device) # Перемещаем функцию потерь на то же устройство, что и модель.
```

```
summary(model_cnn,input_size=(1, 28, 28))
```

Layer (type)	Output Shape	Param #
Conv2d-1	[-1, 6, 24, 24]	156
AvgPool2d-2	[-1, 6, 12, 12]	0
Conv2d-3	[-1, 16, 8, 8]	2,416
AvgPool2d-4	[-1, 16, 4, 4]	0
Linear-5	[-1, 16]	4,112
Linear-6	[-1, 10]	170
Total params: 6,854		
Trainable params: 6,854		
Non-trainable params: 0		
Input size (MB): 0.00		
Forward/backward pass size (MB): 0.04		
Params size (MB): 0.03		
Estimated Total Size (MB): 0.07		

Можем (в среднем) использовать те же функции что и раньше

(не забываем заменить модель)

```
EPOCHS = 10

best_valid_loss = float('inf')

for epoch in range(EPOCHS):

    start_time = time.monotonic()

    # обучаем на тренировочных данных
    train_loss, train_acc = train(model_cnn, train_iterator, optimizer, criterion, device)
    # оцениваем на валидационных
    valid_loss, valid_acc = evaluate(model_cnn, test_iterator, criterion, device)

    # сохраняем лучшую модель по потерям на валидационной выборке
    if valid_loss < best_valid_loss:
        best_valid_loss = valid_loss
        torch.save(model_cnn.state_dict(), 'tut1-model_cnn.pt')

    end_time = time.monotonic()

    epoch_mins, epoch_secs = epoch_time(start_time, end_time)

    print(f'Epoch: {epoch+1:02} | Epoch Time: {epoch_mins}m {epoch_secs}s')
    print(f'\tTrain Loss: {train_loss:.3f} | Train Acc: {train_acc*100:.2f}%')
    print(f'\tVal. Loss: {valid_loss:.3f} | Val. Acc: {valid_acc*100:.2f}%')
```

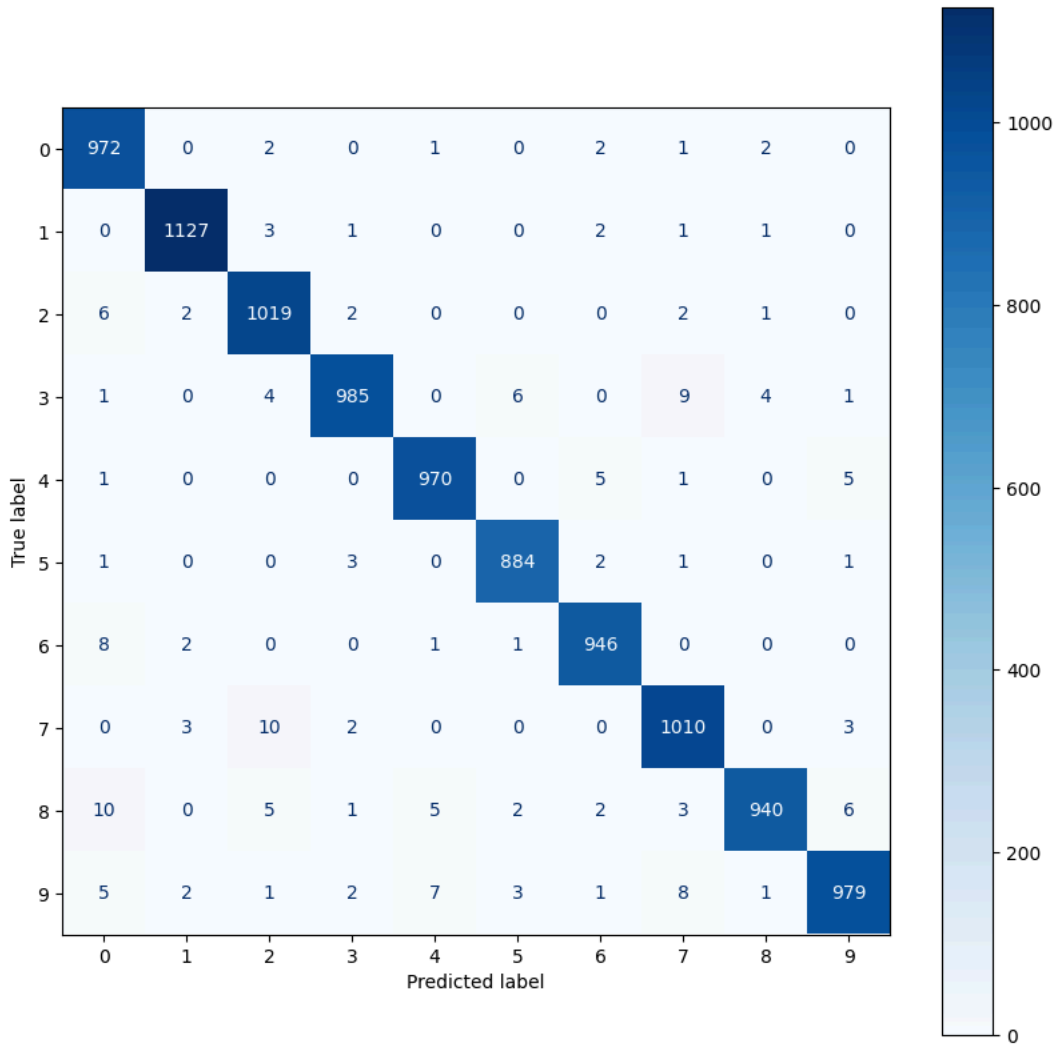
100%

10/10 [03:29<00:00, 19.53s/it]

Epoch: 01 | Epoch Time: 0m 23s
Train Loss: 1.275 | Train Acc: 67.20%
Val. Loss: 0.598 | Val. Acc: 87.23%

```
images, labels, probs = get_predictions(model_cnn, test_iterator, device)
# находим предсказание модели как аргмакс вероятности
pred_labels = torch.argmax(probs, 1)
```

```
plot_confusion_matrix(labels, pred_labels)
```



Epoch: 10 | Epoch Time: 0m 18s
Paraziterno, no v srednem odinakove segitna PyTorch i TensorFlow vedut seba +- odinakovo
Train Loss: 0.074 | Train Acc: 97.90%
Val. Loss: 0.061 | Val. Acc: 98.33%

Итоги блокнота: Чему мы научились?

В этом блокноте мы глубоко погрузились в основы работы с PyTorch для решения задачи классификации изображений на примере датасета MNIST. Вот ключевые моменты, которые мы изучили:

- Воспроизводимость результатов:** Важность установки случайных начальных чисел (`seed`) для Python, NumPy и PyTorch (`torch_seed`) для обеспечения повторяемости экспериментов.
- Работа с данными:** Загрузка датасета MNIST с использованием `torchvision.datasets`, понимание структуры данных (изображение в формате PIL и метка).
- Предобработка данных:** Вычисление среднего (`mean`) и стандартного отклонения (`std`) для нормализации пикселей. Создание конвейера преобразований (`transforms.Compose`) с использованием `ToTensor()` (для преобразования в тензор и масштабирования в [0,1]) и `Normalize()` (для приведения к нулевому среднему и единичной дисперсии).
- Загрузчики данных (DataLoader):** Создание и использование `DataLoader` для эффективной итерации по данным батчами, с опциями перемешивания (`shuffle`) для тренировочных данных.
- Полносвязная нейронная сеть (Sequential API):**
 - Построение простой полносвязной модели с использованием `nn.Sequential`.
 - Использование слоев `nn.Flatten()` (для выпрямления изображений) и `nn.Linear()` (полносвязные слои).
 - Применение функции активации `Sigmoid`.
 - Получение сводки модели с `torchsummary.summary()`.
 - Инициализация оптимизатора (`optim.SGD`) и функции потерь (`nn.CrossEntropyLoss`).
- Сверточная нейронная сеть (Class-based API):**
 - Создание более сложной модели (`NetLeNet`) как класса, наследующегося от `nn.Module`.
 - Определение слоев (`nn.Conv2d`, `nn.AvgPool2d`, `nn.Linear`) в методе `__init__`.
 - Реализация логики прямого прохода (`forward`) через сеть, демонстрируя последовательность операций свертки, активации и пулинга.
 - Расчет размерностей входных признаков для полносвязных слоев после сверточных.
 - Использование другого оптимизатора (`torch.optim.Adam`).
- Цикл обучения и оценки:** Разработка функций `train()` и `evaluate()` для структурированного обучения и проверки модели.
 - Использование `model.train()` и `model.eval()` для переключения режимов модели.
 - Обнуление градиентов (`optimizer.zero_grad()`) выполнение прямого прохода, вычисление потерь, обратного